



CENTRO UNIVERSITÁRIO DE BRASÍLIA - CEUB
CIÊNCIA DA COMPUTAÇÃO

LUCAS GONÇALVES BALDUINO - 22409139

INTELIGÊNCIA ARTIFICIAL- LABORATÓRIO 2

BRASÍLIA

2026

1. INTRODUÇÃO

Esse relatório descreve o estudo e a implementação de três dos principais algoritmos de aprendizado de máquina supervisionado: Árvore de Decisão, Regressão Logística e Máquina de Vetores de Suporte (SVM). O objetivo é compreender o funcionamento de cada algoritmo e compreender suas implementações.

2. ÁRVORE DE DECISÃO

A Árvore de Decisão é um algoritmo de aprendizado supervisionado utilizado para tarefas de classificação e regressão. Estruturalmente, o modelo assemelha-se a um fluxograma ou árvore binária, composta por raízes, nós de decisão, galhos e folhas.

2.1 Funcionamento

O algoritmo realiza o treinamento utilizando os nós internos como estruturas de decisão que representam testes em atributos. Cada teste resulta em uma ramificação, culminando nos nós folha, que definem a classe final ou o valor previsto. Para otimizar a estrutura da árvore, utilizam-se dois métodos principais:

- **Ganho de Informação:** Baseia-se no conceito de Entropia. Ele mede a redução da incerteza após a divisão dos dados. Quanto maior o ganho, melhor o atributo para separar os grupos.
- **Índice de Gini:** Mede a probabilidade de um elemento ser classificado incorretamente. Quanto menor o índice, maior a "pureza" do nó e melhor a identificação dos elementos.

2.2 Aplicações

Devido à sua alta interpretabilidade, as Árvores de Decisão são amplamente aplicadas em sistemas de aprovação de crédito bancário, diagnósticos médicos e detecção de fraudes financeiras.

2.3 Implementação

Python

```
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
```

Função para importar os dados

```
def importdata():
    url = "https://archive.ics.uci.edu/static/public/12/data.csv"
    balance_data = pd.read_csv(url, header=0)
```

```

return balance_data

# Divisao do dataset
def splitdataset(balance_data):
    X = balance_data.iloc[:, 1:5].values
    Y = balance_data.iloc[:, 0].values
    return train_test_split(X, Y, test_size=0.3, random_state=100)

# Treinamento
data = importdata()
X_train, X_test, y_train, y_test = splitdataset(data)

clf_gini = DecisionTreeClassifier(criterion="gini", random_state=100, max_depth=3)
clf_gini.fit(X_train, y_train)

y_pred = clf_gini.predict(X_test)
print(f"Acurácia Árvore de Decisão: {accuracy_score(y_test, y_pred)*100:.2f}%")

```

3. REGRESSÃO LOGÍSTICA

A Regressão Logística é um algoritmo supervisionado focado em problemas de classificação. Diferente da regressão linear, que prevê valores contínuos, este modelo foca na probabilidade de uma instância pertencer a uma categoria específica.

3.1 Funcionamento

O algoritmo utiliza a **Função Sigmoid** para mapear qualquer valor real em um intervalo entre 0 e 1. Existem três variações principais:

- **Binomial:** Saída em duas classes (ex: Sim/Não).
- **Multinomial:** Mais de duas classes sem ordem (ex: Gato/Cão/Pássaro).
- **Ordinal:** Classes que possuem uma hierarquia (ex: Baixo/Médio/Alto).

3.2 Requisitos do Modelo

Para uma implementação eficiente, as observações devem ser independentes, o conjunto de dados deve estar livre de *outliers* significativos e, preferencialmente, deve haver um volume substancial de dados para garantir a consistência estatística.

3.3 Implementação

```

Python
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

```

```
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=23)

clf = LogisticRegression(max_iter=10000, random_state=0)
clf.fit(X_train, y_train)

acc = accuracy_score(y_test, clf.predict(X_test)) * 100
print(f"Acurácia Regressão Logística: {acc:.2f}%")
```

4. MÁQUINA DE VETORES DE SUPORTE (SVM)

O *Support Vector Machine* (SVM) é um algoritmo de aprendizado supervisionado robusto utilizado tanto para classificação quanto para regressão, embora seja mais proeminente em tarefas de classificação complexas.

4.1 Funcionamento

O objetivo central do SVM é encontrar um **hiperplano** em um espaço N-dimensional que separe os dados de diferentes classes com a **margem máxima**.

- **Vetores de Suporte:** São os pontos de dados mais próximos do hiperplano e que influenciam sua posição e orientação.
- **Margem:** É a distância entre o hiperplano e os vetores de suporte. O SVM busca maximizar essa distância para garantir que o modelo tenha boa capacidade de generalização.
- **Kernel Trick:** Quando os dados não são linearmente separáveis no espaço original, o SVM utiliza funções de *kernel* para projetar os dados em uma dimensão superior onde uma separação linear se torna possível.

4.2 Aplicações

O SVM é altamente eficaz em problemas de alta dimensionalidade, sendo comumente aplicado em reconhecimento de caligrafia, classificação de imagens, bioinformática (classificação de proteínas) e análise de sentimentos.

4.3 Implementação

```
Python
from sklearn.datasets import load_breast_cancer
import matplotlib.pyplot as plt
from sklearn.inspection import DecisionBoundaryDisplay
from sklearn.svm import SVC

# Carregamento de dados
cancer = load_breast_cancer()
X = cancer.data[:, :2] # Usando apenas duas features para visualização
y = cancer.target
```

```
# Treinamento do Modelo SVM com Kernel Linear
svm = SVC(kernel="linear", C=1)
svm.fit(X, y)

# Visualização das Fronteiras de Decisão
DecisionBoundaryDisplay.from_estimator(
    svm,
    X,
    response_method="predict",
    alpha=0.8,
    cmap="Pastel1",
    xlabel=cancer.feature_names[0],
    ylabel=cancer.feature_names[1],
)

plt.scatter(X[:, 0], X[:, 1], c=y, s=20, edgecolors="k")
plt.title("Fronteira de Decisão - SVM")
plt.show()
```

5. CONCLUSÃO

A escolha do algoritmo ideal depende intrinsecamente da natureza dos dados e do objetivo do projeto. A **Árvore de Decisão** destaca-se pela transparência no processo decisório. A **Regressão Logística** é eficiente e rápida para probabilidades binárias em dados linearmente separáveis. Por fim, o **SVM** demonstra sua força em cenários complexos e de alta dimensionalidade, onde a margem de separação precisa ser otimizada. A prática laboratorial permitiu observar que, embora as acurácias possam ser similares em datasets controlados, o custo computacional e a explicabilidade variam significativamente entre os modelos.

REFERÊNCIAS

GEEKSFORGEES. **Decision Tree Introduction with Examples**. Disponível em: <https://www.geeksforgeeks.org/machine-learning/decision-tree-introduction-example/>. Acesso em: 29 abr. 2026.

GEEKSFORGEES. **Decision Tree Implementation in Python**. Disponível em: <https://www.geeksforgeeks.org/machine-learning/decision-tree-implementation-python/>. Acesso em: 29 abr. 2026.

GEEKSFORGEES. **Understanding Logistic Regression**. Disponível em: <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>. Acesso em: 29 abr. 2026.

GEEKSFORGEEKS. **Support Vector Machine (SVM) Algorithm**. Disponível em: <https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/>. Acesso em: 29 abr. 2026.